

# MyST syntax

Turning on the MyST pipeline gives you cross references, numbered figures, directives, and roles, and keeps every executable block working.

Shravan Goswami / <https://shravangoswami.com>

Almanac can parse your Markdown into a **MyST** document tree instead of straight to HTML. That buys you the things technical writing needs and Markdown has no syntax for: references that write their own text, figures that number themselves, and directives.

It is off by default. Turn it on with a future flag:

```
// astro.config.mjs
almanac({
  title: "My Project",
  future: { myst: true },
})
```

MyST is a superset of Markdown, so nothing you already wrote stops working.

## Install the toolchain

The MyST packages are optional peer dependencies, so a project that does not use them does not pay for them:

```
npm install myst-parser myst-to-html myst-transforms unified vfile rehype-stringify
```

Turning the flag on without them fails the build with a message naming the package that is missing, rather than silently rendering something different.

## Cross references

Label anything with (name)= on the line above it, then link to it with an empty link text. The text is filled in from whatever you referenced:

```
(setup)=
## Setting things up
```

Later on, see `[](#setup)` and the reader gets "Setting things up".

Reference a figure and you get its number instead. Write your own text and it wins, so `[the setup guide]` (`#setup`) still says what you wrote.

A reference to a label that does not exist warns during the build and renders as visible text rather than an invisible empty link, so a broken reference is obvious on the page instead of silently disappearing.

## Numbered figures

```
:::{figure} ./chart.png
:label: fig-growth
:alt: Growth over time
```

```
Requests per second, measured weekly.
:::
```

The caption is prefixed with “Figure 1”, numbered in document order, and `[](#fig-growth)` renders as a link reading “Figure 1”. Relative image paths go through Astro’s asset pipeline exactly as they do in plain Markdown mode.

## Directives and roles

Admonitions become directives rather than raw HTML:

```
 :::{note}
 Directive bodies are full Markdown, so bold and [links](#setup) work.
 :::
```

note, tip, warning, caution, important, and danger are all available. Inline roles cover smaller things, such as {abbr}`MyST` (Markedly Structured Text) for an abbreviation with a tooltip.

## Math

Inline  $E = mc^2$  and display blocks:

```
 $$
 \int_0^1 x^2 \, dx = \frac{1}{3}
 $$
```

Almanac emits math-inline and math-display elements holding the source. It does not ship a math renderer, so add KaTeX or MathJax yourself through head if you want typeset output.

## Executable blocks keep working

Both syntaxes run when future.execute is also on. MyST's own is {code-cell}:

```
 ```{code-cell} js
 console.log("runs at build time");
 ```
```

And the ordinary fence with exec behaves identically, so content moves between the two modes without being rewritten:

```
 ```js exec
 console.log("also runs at build time");
 ```
```

That works because the fence directives are recovered from the source line the block starts on. MyST's parser keeps a block's language but discards the rest of the info string, so exec, hide-code, and timeout= would otherwise be lost.

## What is different in MyST mode

Worth knowing before you flip the flag:

- **Remote images are not optimized.** Local paths still are. Deciding whether a remote URL is allowed needs one of Astro's internal helpers, which a strict package manager will not expose to a dependency, and claiming an image Astro then refuses would fail the build. Remote images render as ordinary <img> tags.
- **Syntax highlighting is unchanged.** Code blocks go through the same Shiki step and honour your shikiConfig.
- **Raw HTML still passes through**, so existing <div class="callout note"> markup keeps rendering.
- **smartyants and gfm toggles do not apply.** Those are options on Astro's remark pipeline. MyST does its own typography and always supports tables, strikethrough, and task lists.

## What is not built yet

Citations parse but there is no bibliography renderer: {cite} needs a .bib file and a citation style, and neither is wired up. Cross-document references, the ones that point into another page, resolve within a page only. Both need the cross-collection content phase that docs versioning also needs.